

STOCKPILOT AI

Compte rendu de progression - Jour 1

Session du 3 au 4 août 2026

5 documents de cadrage	7 tables PostgreSQL	5 schémas de données	175 200 snapshots d'inventaire
-------------------------------------	-------------------------------	--------------------------------	---

Objectif de la journée

Passer d'une idée de SaaS à une fondation technique reproductible : cadrage, Git, Docker, PostgreSQL et données synthétiques réalistes.

Dépôt : github.com/Tahamahir/stockpilot-ai

1. Résumé exécutif

Résultat principal. La première journée a permis de transformer StockPilot AI en un projet structuré, versionné et exécutable. Le périmètre du MVP est défini, l'architecture cible est documentée, PostgreSQL fonctionne dans Docker et le modèle relationnel initial est opérationnel.

Fondation Data. Les tables métier centrales ont été créées avec leurs contraintes d'intégrité et leurs index. Un jeu de données minimal a validé les relations SQL, puis un générateur de données synthétiques réalistes a été mis en place pour préparer les futurs pipelines ETL et les modèles de prévision.

Prochaine priorité. Le prochain jalon est l'intégration d'Apache Airflow afin de détecter, contrôler et charger automatiquement les fichiers CSV dans le schéma raw.

État d'avancement à la fin du Jour 1

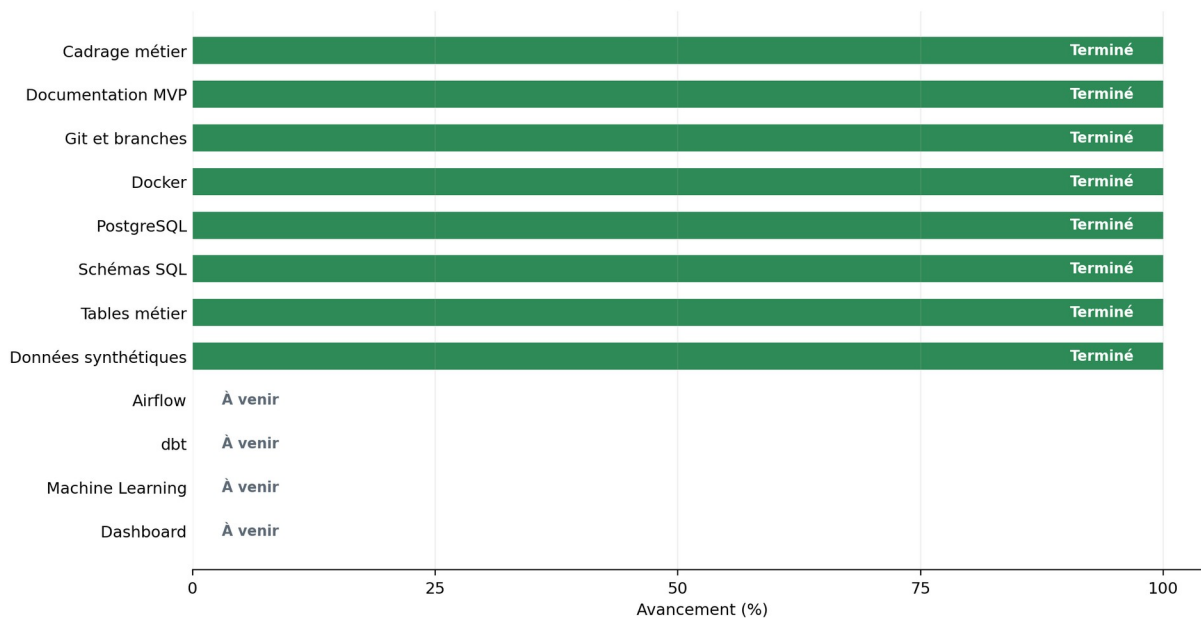


Figure 1 - Avancement global après la première session de travail.

2. Travaux réalisés

2.1 Cadrage métier et produit

- Définition du client cible initial : petites et moyennes entreprises gérant leurs ventes et stocks principalement dans Excel ou CSV.
- Formulation de la question métier centrale : quels produits commander, en quelle quantité et pourquoi ?
- Définition du périmètre du MVP, des fonctionnalités incluses et des éléments reportés après validation.
- Rédaction des user stories pour le gérant, le responsable des achats, le responsable de stock et l'analyste.
- Création du dictionnaire de données et de l'architecture technique cible.

2.2 Fichiers de documentation créés

Fichier	Rôle
---------	------

docs/business_problem.md	Problème métier, client cible et proposition de valeur.
docs/mvp_scope.md	Fonctionnalités incluses, exclusions et critères de réussite.
docs/user_stories.md	Besoins utilisateurs et critères d'acceptation.
docs/data_dictionary.md	Définition initiale des entités et colonnes.
docs/architecture.md	Flux de données, rôle des outils et trajectoire vers le SaaS.

2.3 Gestion du code avec Git

Le dépôt GitHub a été initialisé avec les branches **main**, **develop** et une branche fonctionnelle pour le générateur de données. Les commits sont organisés par objectif, ce qui prépare les futurs pull requests et la CI/CD.

Repository : <https://github.com/Tahamahir/stockpilot-ai.git>
Branches : main, develop, feature/synthetic-data-generator
État : working tree clean et branches synchronisées

3. Architecture technique retenue

L'architecture cible sépare clairement ingestion, transformation, modélisation, exposition des résultats et interface utilisateur. Cette séparation rend le système testable, remplaçable par module et évolutif vers une offre multi-entreprises.

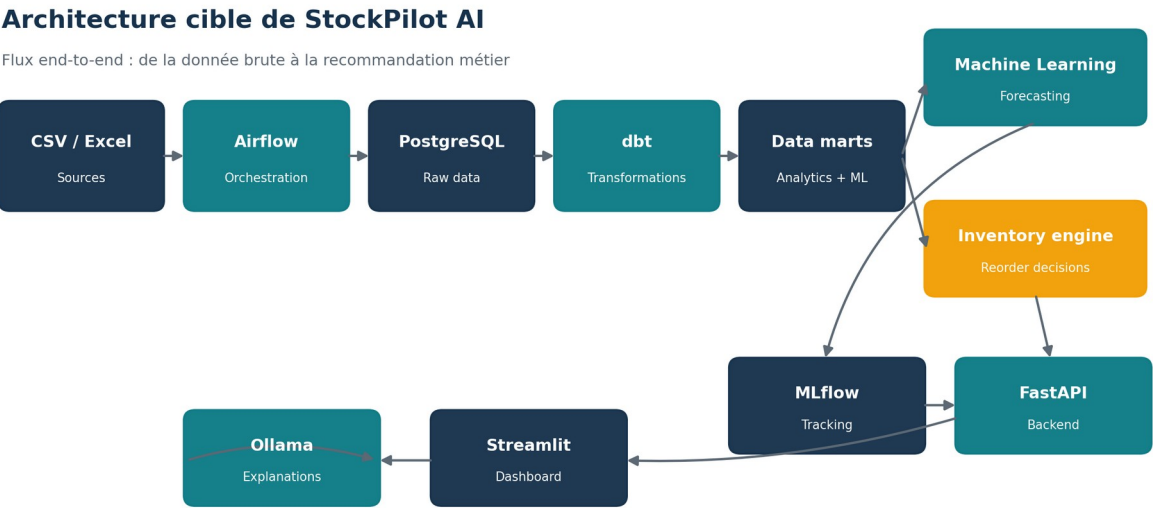


Figure 2 - Architecture end-to-end prévue pour le MVP.

Composant	Responsabilité principale
-----------	---------------------------

Apache Airflow	Orchestrer les imports, contrôles, transformations et entraînements.
PostgreSQL	Stocker les données raw, staging, analytics et ML.
dbt	Nettoyer, tester, documenter et produire les data marts.
MLflow	Tracer les expériences et versionner les modèles.
FastAPI	Exposer les prévisions, alertes et recommandations.
Streamlit	Présenter les indicateurs et décisions métier.
Ollama	Expliquer les résultats sans inventer les calculs.

4. Environnement Docker et PostgreSQL

4.1 Validation de Docker

Docker Desktop a d'abord rencontré un blocage du moteur Linux. Le problème a été résolu en redémarrant correctement Docker/WSL, puis le service PostgreSQL a été lancé avec succès.

```
Docker version      : 28.3.2
Docker Compose version: v2.39.1-desktop.1
Image PostgreSQL    : postgres:17-alpine
Port local          : 5432
État du conteneur   : Up (healthy)
```

4.2 Schémas PostgreSQL créés

La base stockpilot est organisée en cinq couches pour éviter de mélanger les données brutes, les transformations et les résultats analytiques :

raw	staging	intermediate	analytics	ml
-----	---------	--------------	-----------	----

4.3 Tables métier opérationnelles

Les tables companies, stores, suppliers, products, sales et inventory sont créées avec des clés étrangères, des contraintes de contrôle, des index et une logique tenant_id préparant l'évolution vers un SaaS multi-entreprises.

Modèle relationnel initial

Les clés composées tenant_id + identifiant préparent l'isolation multi-entreprises.

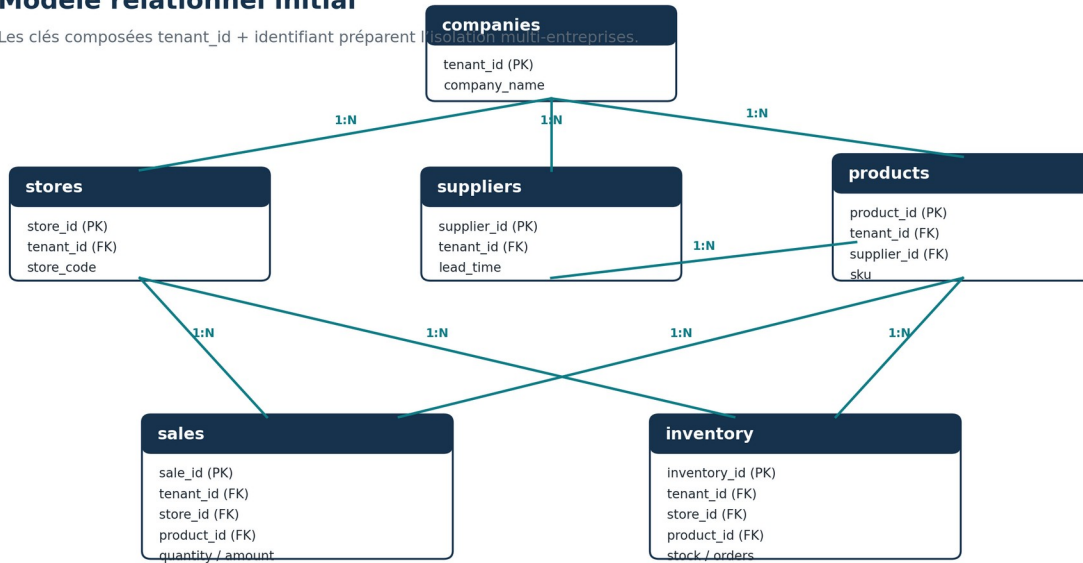


Figure 3 - Modèle relationnel initial de la couche raw.

```

raw.companies
raw.stores
raw.suppliers
raw.products
raw.sales
raw.inventory
raw.pipeline_test
  
```

5. Données de démonstration

5.1 Deux niveaux de données

Données minimales SQL. Elles ont servi à vérifier les clés étrangères, les contraintes et les premières requêtes métier, notamment le calcul de la position de stock.

Données synthétiques réalistes. Un générateur Python reproductible a ensuite été créé pour simuler des volumes suffisants pour Airflow, dbt, le Machine Learning et la démonstration du produit.

Configuration du dataset synthétique de démonstration



Figure 4 - Configuration officielle du dataset synthétique.

5.2 Comportements simulés

- saisonnalité hebdomadaire et annuelle ;
- tendance croissante ou décroissante selon le produit ;
- effet des promotions et des remises ;
- produits à forte rotation, faible rotation et demande intermittente ;
- ruptures de stock, demande non satisfaite et backorders ;
- délais fournisseurs et retards de livraison ;
- réapprovisionnement par quantité minimale et taille de colis ;
- fichier invalide séparé pour tester les futurs contrôles ETL.

5.3 Limite méthodologique

Point de vigilance : les données synthétiques sont adaptées au développement et à la démonstration, mais elles ne prouvent pas à elles seules la performance commerciale. Une version vendable devra être validée par backtesting sur les données d'une entreprise pilote.

6. Validations techniques effectuées

Élément	Statut	Preuve / résultat
Git	VALIDÉ	Dépôt initialisé, branches distantes créées et état propre.
Docker	VALIDÉ	Client et moteur opérationnels après résolution du blocage WSL.
PostgreSQL	VALIDÉ	Conteneur actif et healthcheck au statut healthy.
SQL	VALIDÉ	Schémas, tables, contraintes, clés étrangères et index créés.
Données	VALIDÉ	Données de test insérées et requêtes métier validées.
Python	VALIDÉ	Générateur synthétique compilé et exécuté avec seed

reproductible.

7. Incident rencontré et résolution

Incident. Docker Compose ne pouvait pas télécharger l'image PostgreSQL, car le canal dockerDesktopLinuxEngine était indisponible. Docker Desktop restait bloqué au démarrage pendant plus de trente minutes.

Diagnostic. Le client Docker était installé, mais le moteur Linux n'était pas actif.

Résolution. Arrêt de la commande, redémarrage de Docker Desktop/WSL, puis nouvelle exécution de Docker Compose. Le conteneur PostgreSQL a ensuite atteint le statut healthy.

8. Livrables disponibles à la fin du Jour 1

- Repository GitHub structuré avec main, develop et branche fonctionnelle.
- Documentation complète du problème métier et du MVP.
- Architecture technique cible documentée.
- docker-compose.yml avec PostgreSQL 17.
- Scripts SQL d'initialisation, création des tables et données de démonstration.
- Modèle relationnel multi-tenant initial.
- Générateur Python de données commerciales synthétiques.
- Configuration reproductible : 730 jours, 120 produits, 2 magasins et 8 fournisseurs.
- Jeu de fichiers volontairement invalides pour les futurs tests de qualité.

9. Prochaines étapes - Jour 2

1	Ajouter Apache Airflow au Docker Compose.
2	Créer la connexion Airflow vers PostgreSQL.
3	Développer un DAG de détection et validation des fichiers CSV.
4	Charger les enregistrements valides dans raw et isoler les lignes rejetées.
5	Journaliser les erreurs dans raw.rejected_records.
6	Ajouter les premiers tests automatisés du pipeline d'ingestion.

Conclusion : le projet dispose désormais d'une base saine pour démarrer la Data Engineering. Le cœur du Jour 2 sera l'automatisation de l'ingestion, sans encore introduire dbt ni les modèles de prévision.

Note de traçabilité : ce compte rendu synthétise les commandes, scripts et validations de la session. Aucune performance Machine Learning n'a encore été mesurée.

STOCKPILOT AI

Compte rendu de progression - Jour 2

Session du 4 août 2026

3 DAGs Airflow	11 tâches d'ingestion	8 fichiers CSV	4 rejets détectés
--------------------------	---------------------------------	--------------------------	-----------------------------

Objectif de la journée

Automatiser la validation et le chargement des données commerciales dans PostgreSQL, puis tracer les enregistrements invalides sans introduire la couche de transformation analytique.

Dépôt : github.com/Tahamahir/stockpilot-ai

1. Résumé exécutif

Résultat principal. Le Jour 2 a permis de rendre l'ingestion des données reproductible et observable. Apache Airflow est désormais exécuté dans Docker, connecté à PostgreSQL et capable de vérifier l'environnement, de charger les huit fichiers synthétiques dans le schéma raw et de contrôler les volumes chargés.

Qualité des données. Un pipeline distinct analyse un jeu volontairement invalide et journalise chaque rejet dans raw.rejected_records avec le fichier source, le numéro de ligne, le type d'erreur, le message et le dag_run_id.

Résultat opérationnel. Le run complet du DAG d'ingestion a terminé avec succès en 43,088 secondes. Les 175 200 snapshots d'inventaire ont été chargés, et les quatre anomalies prévues ont été correctement détectées.

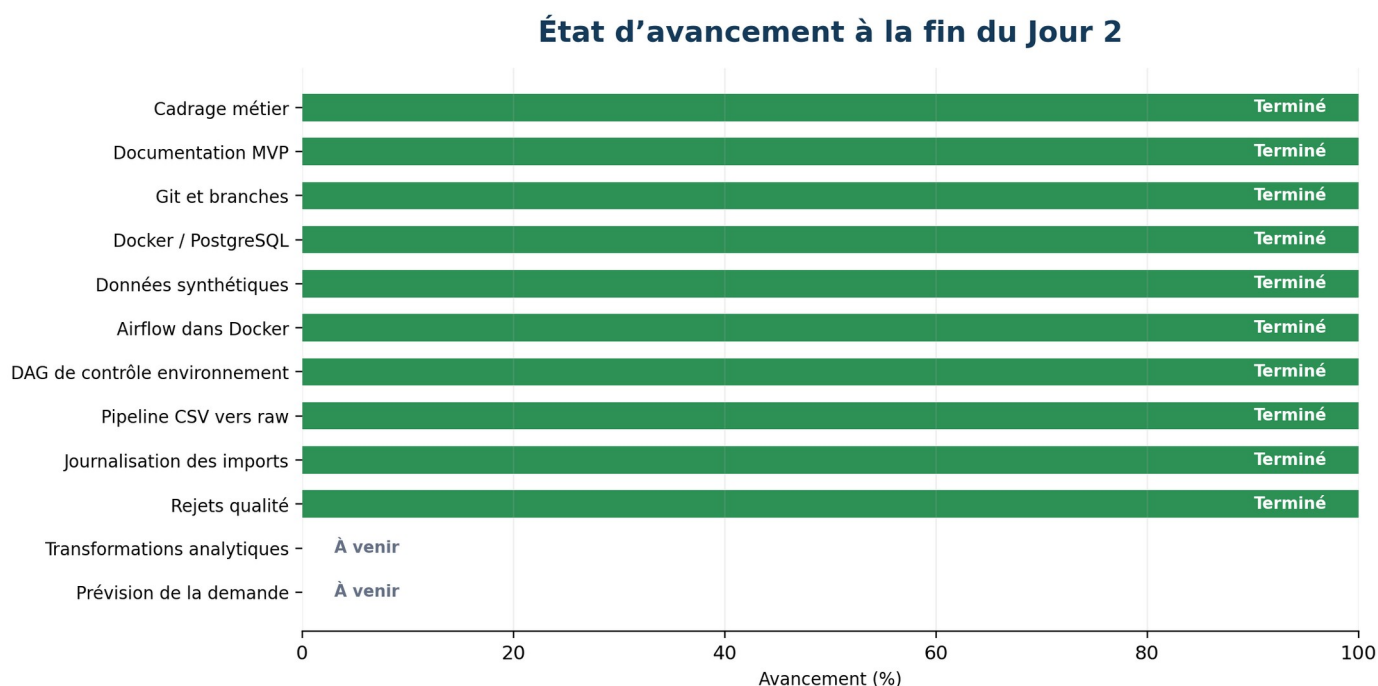


Figure 8 - Avancement global après la deuxième session de travail.

2. Périmètre du Jour 2

- Conteneurisation d'Apache Airflow avec une base PostgreSQL dédiée aux métadonnées.
- Connexion sécurisée d'Airflow vers la base métier StockPilot.
- Création d'un DAG de contrôle de l'environnement.
- Création du pipeline d'ingestion CSV vers le schéma raw.
- Ajout de la journalisation des imports et des rejets qualité.
- Validation des erreurs contrôlées et correction des incidents techniques rencontrés.

3. Mise en place d'Apache Airflow

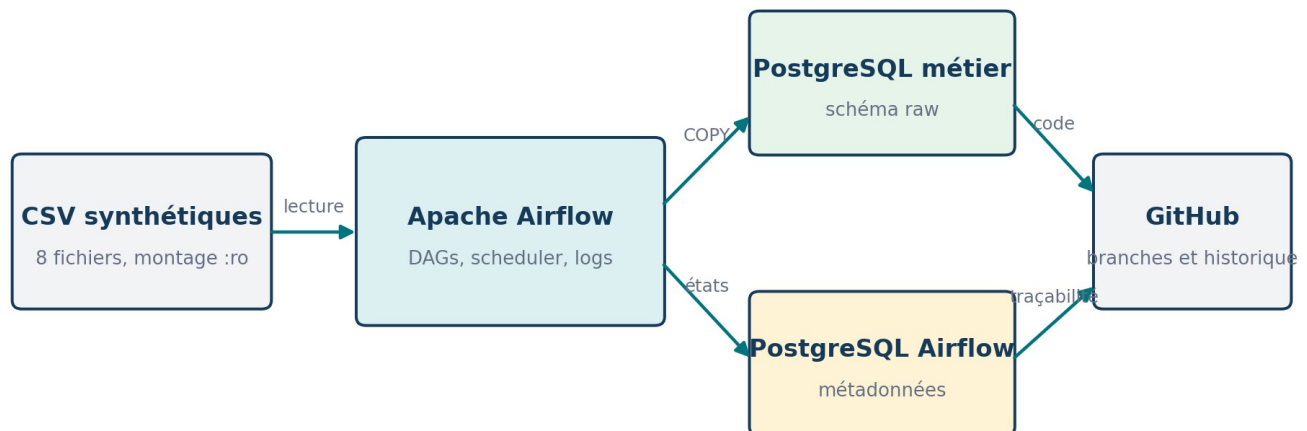
3.1 Services Docker opérationnels

Trois services ont été démarrés et validés : la base métier stockpilot-postgres, la base de métadonnées stockpilot-airflow-db et le service stockpilot-airflow. Les deux bases ont atteint le statut healthy et l'interface Airflow a été exposée sur le port 8080.

Service	Rôle	Validation
stockpilot-postgres	Données commerciales et schémas raw/staging/intermediate/analytics/ml	Healthy
stockpilot-airflow-db	Métadonnées des DAGs, runs et tâches	Healthy
stockpilot-airflow	Interface, scheduler, processeur de DAGs et logs	Démarré

3.2 Architecture obtenue

Architecture Data Engineering mise en place au Jour 2



Les données sources restent en lecture seule ; une copie temporaire /tmp est utilisée pour les chargements COPY.

Figure 9 - Architecture Docker et circulation des données au Jour 2.

3.3 Sécurité et reproductibilité

- Les secrets restent dans le fichier .env, exclu de Git.
- Le dossier des données est monté en lecture seule dans Airflow.
- Les connexions sont injectées par variables d'environnement.
- Les volumes Docker conservent les bases après l'arrêt du PC.
- Le fichier .dockerignore limite le contexte de build et évite l'envoi de fichiers volumineux.

4. Premier DAG : contrôle de l'environnement

Le DAG `stockpilot_environment_check` constitue le test de santé du projet. Il vérifie que les huit fichiers CSV générés existent, qu'ils ne sont pas vides, que la connexion PostgreSQL fonctionne et que les cinq schémas attendus sont disponibles.

4.1 Contrôles exécutés

Tâche	Contrôles	Résultat
<code>check_generated_files</code>	Présence, taille et disponibilité des 8 CSV	Succès
<code>check_stockpilot_database</code>	Base, utilisateur, schémas et nombre de tables raw	Succès

4.2 Intégration Git et branches

Le travail Airflow a été développé sur des branches fonctionnelles puis intégré dans `develop`. Les branches utilisées ont notamment été `feature/airflow-setup`, `feature/raw-data-ingestion` et `feature/data-quality-rejections`.

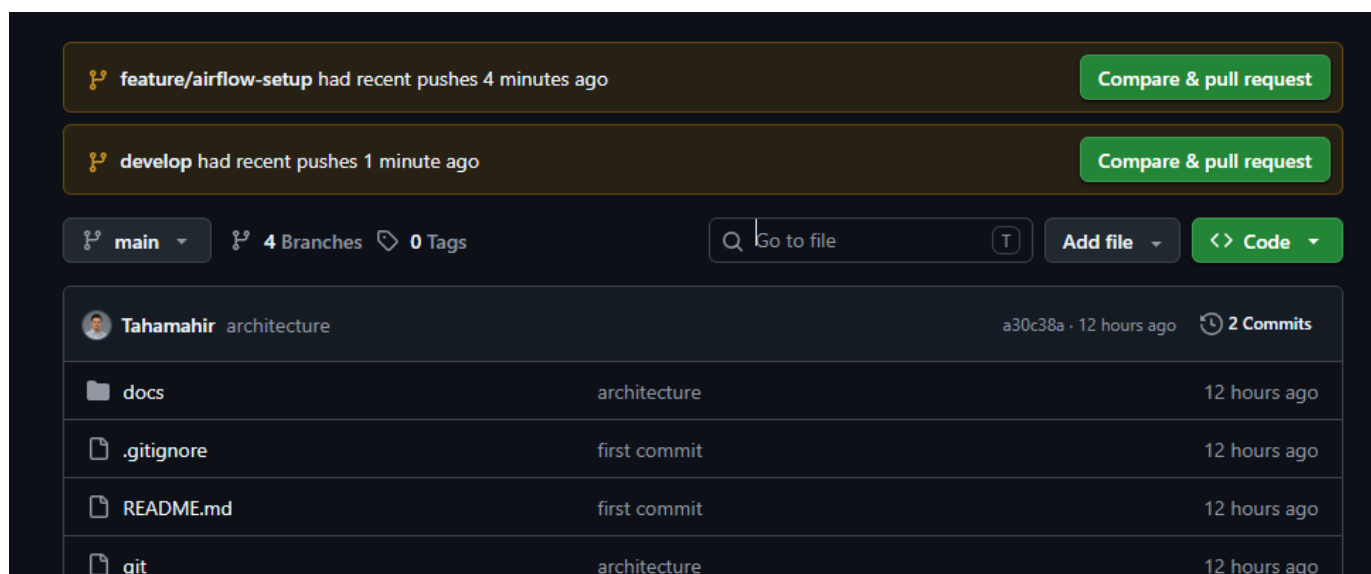


Figure 10 - Suivi des branches fonctionnelles dans le dépôt GitHub.

4.3 Résultat

Ce DAG fournit une vérification rapide après un redémarrage du poste ou une modification de la configuration. Il permet de détecter une absence de fichiers, une base indisponible ou un schéma manquant avant de lancer un traitement plus coûteux.

5. Pipeline d'ingestion CSV vers PostgreSQL

Le DAG stockpilot_raw_data_ingestion automatise le chargement complet du dataset synthétique. Les tâches sont séquentielles afin de respecter les clés étrangères : l'entreprise est chargée avant les magasins et fournisseurs, puis les produits, les promotions, les commandes, les ventes et enfin les snapshots d'inventaire.

DAG stockpilot_raw_data_ingestion



11 tâches séquentielles - 8 fichiers - contrôle final des volumes CSV/PostgreSQL

Figure 11 - Chaîne de traitement du DAG d'ingestion raw.

5.1 Tables de support ajoutées

Table	Utilité
raw.promotions	Périodes promotionnelles et remises par produit/magasin.
raw.purchase_orders	Commandes fournisseurs, dates prévues/réelles et statuts.
raw.ingestion_runs	Volumes détectés, chargés, rejetés, statut et message d'erreur.
raw.rejected_records	Traçabilité des lignes invalides et du run concerné.

5.2 Résultats du run validé

43,088 s durée du run	175 200 lignes inventaire	11 tâches vertes	0 rejet sur fichiers valides
---------------------------------	-------------------------------------	----------------------------	--

La tâche verify_ingestion compare les volumes détectés dans les CSV aux volumes présents dans PostgreSQL. Le run n'est déclaré réussi que lorsque chaque table contient exactement le nombre de lignes attendu.

6. Contrôle qualité et gestion des rejets

Un fichier `sales_invalid.csv` a été généré volontairement pour vérifier le comportement du système face à des données incorrectes. Le DAG `stockpilot_invalid_sales_quality_check` contrôle la structure, les types, les relations avec les référentiels et la cohérence des montants.

6.1 Anomalies détectées

Ligne	Type d'erreur	Diagnostic
2	<code>invalid_quantity</code>	La quantité est négative ou n'est pas strictement positive.
3	<code>invalid_date</code>	La valeur de <code>sale_date</code> ne respecte pas le format ISO.
4	<code>unknown_product</code>	Le produit n'existe pas pour le tenant indiqué.
5	<code>duplicate_sale_reference</code>	La référence de vente est dupliquée dans le fichier.

6.2 Traçabilité enregistrée

- Nom du fichier source et table cible.
- Numéro exact de la ligne rejetée.
- Type et message d'erreur.
- Enregistrement original conservé en JSONB.
- Identifiant `dag_run_id` permettant de distinguer chaque exécution.
- Horodatage de la détection.

6.3 Validation des exécutions répétées

Deux exécutions successives ont d'abord produit huit lignes, soit quatre rejets par run. Le regroupement par `dag_run_id` a confirmé cette séparation. Après nettoyage des données de test et une nouvelle exécution unique, le résultat attendu est de quatre rejets distincts.

Résultat qualité : 4 lignes invalides sur 4 détectées, classifiées et journalisées.

7. Incidents rencontrés et résolutions

Incident	Cause identifiée	Résolution
Build Docker interrompu par unexpected EOF	Contexte de build trop volumineux : .venv, données générées et fichiers locaux.	Création de .dockerignore puis reconstruction propre.
copy_expert : Read-only file system	Le provider ouvrait le CSV en mode r+ alors que le volume data était monté :ro.	Copie temporaire du CSV dans /tmp, chargement COPY, puis suppression du fichier temporaire.
Argument ql inattendu	Faute de frappe dans l'appel copy_expert.	Remplacement de ql par sql et nouveau run.
Commande Airflow avec -d non reconnue	Syntaxe CLI différente dans la version utilisée.	Utilisation du dag_id comme argument positionnel.

8. Preuve d'exécution réussie

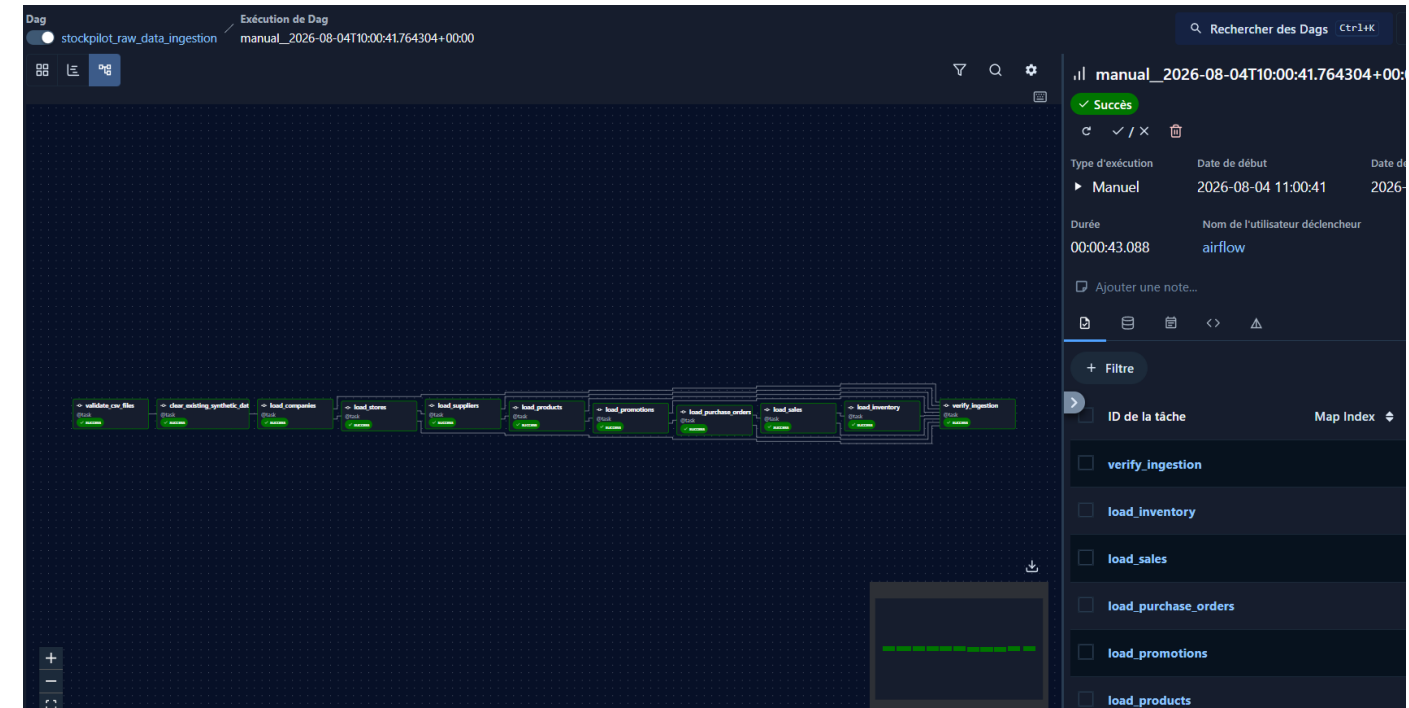


Figure 12 - Run Airflow réussi : toutes les tâches du pipeline sont en succès.

9. Livrables disponibles à la fin du Jour 2

- Image Docker Airflow personnalisée avec le provider PostgreSQL.
- Configuration Docker Compose avec trois services persistants.
- DAG stockpilot_environment_check.
- DAG stockpilot_raw_data_ingestion.
- DAG stockpilot_invalid_sales_quality_check.
- Tables raw.promotions, raw.purchase_orders, raw.ingestion_runs et raw.rejected_records.
- Pipeline de chargement en masse par COPY avec fichiers temporaires sécurisés.
- Journalisation des volumes chargés et des erreurs par run.
- Contrôle de quatre catégories d'anomalies dans les ventes.
- Historique Git sur la branche develop.

10. État technique atteint

Composant	État en fin de session
Docker	Services persistants, redémarrage vérifié après extinction du PC.
Airflow	Interface accessible, DAGs détectés et exécutions traçables.
PostgreSQL	Schéma raw alimenté et contraintes relationnelles respectées.
Ingestion	8 fichiers validés et chargés dans le bon ordre.
Qualité	Rejets isolés avec détail de l'erreur et dag_run_id.
Git	Fonctionnalités enregistrées dans develop.

11. Prochaines étapes

- Intégrer la validation ligne par ligne au pipeline principal.
- Charger les lignes valides tout en isolant les lignes incorrectes dans le même run.
- Ajouter des tests automatisés sur les contraintes de données et les volumes.
- Construire les premières vues SQL propres pour l'analyse des ventes et du stock.
- Documenter les règles de qualité attendues pour chaque fichier client.

Conclusion : StockPilot AI dispose désormais d'une chaîne d'ingestion automatisée, observable et robuste face aux premières erreurs de données. La base Data Engineering du MVP est opérationnelle.

Note de traçabilité : ce document reprend uniquement les travaux réalisés sur Docker, PostgreSQL, Airflow, l'ingestion et la qualité des données au Jour 2.